

O&G · 5 — A physics-informed neural network for pressure-transient analysis

`04_well_test_history_matching.jl` used a closed-form *approximation* to the diffusivity equation. Here we solve the actual dimensionless radial diffusivity PDE directly with a physics-informed neural network, the same technique as `05_physics_informed_neural_networks.jl`, applied to a bounded (closed outer-boundary) reservoir:

$$\frac{\partial p_D}{\partial t_D} = \frac{1}{r_D} \frac{\partial}{\partial r_D} \left(r_D \frac{\partial p_D}{\partial r_D} \right), \quad r_D \in [1, r_{eD}]$$

with a constant-rate inner boundary (Neumann), a no-flow outer boundary (Neumann), and zero initial pressure depletion:

$$\left. \frac{\partial p_D}{\partial r_D} \right|_{r_D=1} = -1, \quad \left. \frac{\partial p_D}{\partial r_D} \right|_{r_D=r_{eD}} = 0, \quad p_D(r_D, 0) = 0$$
$$p_D(1, t_D)$$

— the dimensionless pressure at the wellbore — is exactly the quantity a pressure gauge records during a well test.

Error message

Multiple expressions in one cell

How would you like to fix it?

- [Split this cell into 2 cells](#)
- [Wrap all code in a `begin ... end` block.](#)

```
1 using NeuralPDE, Lux, ModelingToolkit, Optimization, OptimizationOptimisers, Random, Plots
2 import ModelingToolkit: Interval
```

1. Symbolic PDE specification ⇌

Error message

UndefVarError: ModelingToolkit not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

Show stack trace...

```
1 begin
2     reD = 50.0      # dimensionless drainage radius (re / rw)
3     tDmax = 200.0
4
5     ModelingToolkit.@parameters r t
6     ModelingToolkit.@variables p(..)
7     Dt = ModelingToolkit.Differential(t)
8     Dr = ModelingToolkit.Differential(r)
9     Drr = Dr^2
10
11     eq = Dt(p(r, t)) ~ Drr(p(r, t)) + (1 / r) * Dr(p(r, t))
12
13     bcs = [p(r, 0.0) ~ 0.0,
14            Dr(p(1.0, t)) ~ -1.0,
15            Dr(p(reD, t)) ~ 0.0]
16
17     domains = [r ∈ Interval(1.0, reD), t ∈ Interval(0.0, tDmax)]
18 end
```

2. The network ansatz and training problem ↩

Error message

UndefVarError: Random not defined in this notebook.
Suggestion: check for spelling errors or missing imports.
Hint: Random is loaded but not imported in the active module Main.

Show stack trace...

```
1 begin
2     rng_og5 = Random.default_rng()
3     Random.seed!(rng_og5, 2)
4     pde_chain = Chain(Dense(2, 24, tanh), Dense(24, 24, tanh), Dense(24, 1))
5 end
```

Error message

UndefVarError: PhysicsInformedNN not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

```
1 pde_discretization = PhysicsInformedNN(pde_chain, QuadratureTraining())
```

Error message

UndefVarError: @named not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

```
1 begin
2     @named pressure_pde_system = PDESystem(eq, bcs, domains, [r, t], [p(r, t)])
3     prob_pde = discretize(pressure_pde_system, pde_discretization)
4 end
```

3. Train ⇌

Error message

UndefVarError: OptimizationOptimisers not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

```
1 begin
2     pde_losses = Float64[]
3     pde_callback = function (state, loss)
4         push!(pde_losses, loss)
5         return false
6     end
7
8     res_pde = Optimization.solve(prob_pde, OptimizationOptimisers.Adam(0.01);
9     callback=pde_callback, maxiters=2000)
10 end
```

Error message

UndefVarError: plot not defined in this notebook.
Suggestion: check for spelling errors or missing imports.

```
1 plot(pde_losses; xlabel="iteration", ylabel="PDE + BC residual", yscale=:log10,
2     label="training loss")
```

4. The wellbore pressure response — and a sanity check against the early-time analytical solution ⇌

Before the pressure disturbance reaches the outer boundary, the well behaves as if the reservoir were infinite-acting, and $p_D(1, t_D)$ should follow the well-known log approximation

$$p_D(1, t_D) \approx \frac{1}{2} [\ln(t_D) + 0.80907]$$

This gives us an independent check on the trained network that has nothing to do with the training loss itself.

Error message

Another cell defining `pde_chain` contains errors.

```
1 begin
2     phi_pde = pde_discretization.phi
3     final_pde_ps = res_pde.u
4
5     t_grid = 1.0:1.0:150.0
6     pD_wellbore = [first(phi_pde([1.0, ti], final_pde_ps)) for ti in t_grid]
7     pD_analytical = [0.5 * (log(ti) + 0.80907) for ti in t_grid]
8
9     plot(t_grid, pD_analytical; xscale=:log10, label="infinite-acting log
10    approximation", lw=2, xlabel="tD", ylabel="pD(1, tD)")
11    plot!(t_grid, pD_wellbore; xscale=:log10, label="PINN", lw=2, ls=:dash,
12           title="Wellbore pressure response (PINN vs. analytical early-time solution)")
13 end
```

Takeaways ⇄

- The Neumann inner-boundary condition — the actual physical statement "the well produces at a constant rate" — is passed to `NeuralPDE.jl` as literally $\partial p_D / \partial r_D|_{r_D=1} = -1$, no discretized flux calculation required.
- At early times (before the pressure transient reaches r_{eD}), the PINN's wellbore response should track the classical infinite-acting log approximation; at late times it should flatten out as the closed outer boundary is felt — the same **infinite-acting** → **transition** → **boundary-dominated flow** signature every well test interpreter looks for.
- Because the whole reservoir domain was solved in one mesh-free network, extracting the pressure at *any* radius (not just the wellbore) is a free `phi([r, t], p)` evaluation — useful for interference-test analysis at an offset observation well.